

LINQ Sum Method in C# with Examples

In this article, I will discuss the **LINQ Sum Method in C#** with Examples. Please read our previous article discussing the basics of **LINQ Aggregate Functions in C#**. The LINQ Sum() Method belongs to the category of Aggregate Operators. As part of this article, we will discuss the following pointers.

- **What is the LINQ Sum Method?**
- **Example to Understand Sum Method in C#**
- **LINQ Sum Method with Where Extension Method**
- **How do you use the LINQ Sum Method with Predicate?**
- **Handling Nullable Types with Sum Method**
- **LINQ Sum Method with Complex Type in C#**
- **LINQ Sum and Where Extension Methods with Complex Type**
- **When should we use the LINQ Sum Method in C#?**

What is the LINQ Sum Method in C#?

The Sum method in LINQ is a widely used aggregate function that calculates the sum of a sequence of numeric values. It's part of the **System.Linq** namespace and can be applied to arrays, lists, or any other enumerable collections in .NET. The Sum method is available in various overloads, allowing you to sum up different numeric types such as int, double, decimal, etc., and even to sum up a specific numeric property in a collection of objects.

Example to Understand Sum Method in C#:

Let us understand the LINQ Sum() method with Examples using C#. Using query and method syntax, the following example calculates the sum of all integers in the integer collection. The point that you need to remember is that the Sum Method will work with numeric values only. We don't have any operator called sum in the LINQ Query Syntax. So here, we need to use mixed syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 30, 50, 40, 60, 20, 70, 90, 80,
            //Using Method Syntax
            int MSTotal = intNumbers.Sum();
            //Using Query Syntax
            int QSTotal = (from num in intNumbers
                           select num).Sum();
            Console.WriteLine("Sum = " + QSTotal);
            Console.ReadKey();
        }
    }
}
```

Output: **Sum = 550**

Example to Understand LINQ Sum Method with Where Extension Method using C#

Let us see an example to Understand how we can use the LINQ Sum and the Where Extension Method in C# using both Method and Query Syntax. Our requirement is to calculate the sum of all numbers greater than 50. The following example code exactly does the same.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 30, 50, 40, 60, 20, 70, 90, 80,
            //Using Method Syntax
            int MSTotal = intNumbers.Where(num => num > 50).Sum();
            //Using Query Syntax
            int QSTotal = (from num in intNumbers
                           where num > 50
                           select num).Sum();
            Console.WriteLine("Sum = " + QSTotal);
            Console.ReadKey();
        }
    }
}
```

Output: **Sum = 400**

Example to Understand How to use LINQ Sum Method with Predicate in C#

Let us see an example of using the LINQ Sum Method with Predicate in C# using both Method and Query Syntax. Instead of using the Where Extension method to filter the data, we can also use the other overloaded version of the Sum method, which takes a Predicate as a parameter. Within that predicate, we can write the logic to filter the data. In the below example, within the Sum Method, we are using a Predicate and providing the condition of whether the number is greater than 50. If the number is greater than 50, then we are returning true; else, we are returning false. The following example will calculate the sum of all integers greater than 50.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 10, 30, 50, 40, 60, 20, 70, 90, 80,
```

```

//Using Method Syntax with a Predicate
int MSTotal = intNumbers.Sum(num => {
    if (num > 50)
        return num;
    else
        return 0;
});
//Using Query Syntax with a Predicate
int QSTotal = (from num in intNumbers
               select num).Sum(num => {
    if (num > 50)
        return num;
    else
        return 0;
});
Console.WriteLine("Sum = " + QSTotal);
Console.ReadKey();
}
}
}

```

Output: **Sum = 400**

Handling Nullable Types

The Sum method also has overloads for handling nullable numeric types (int?, double?, etc.). When summing nullable types, the method will ignore null values.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int?[] nullableNumbers = { 1, 2, null, 4, 5 };
            int sum = nullableNumbers.Sum().GetValueOrDefault();

            Console.WriteLine("Sum = " + sum);
            Console.ReadKey();
        }
    }
}

```

Output: **Sum = 12**

Example to Understand LINQ Sum Method with Complex Type in C#:

The Sum method becomes even more powerful when you need to sum a specific property of objects in a collection. For example, if you have a collection of objects and each object has a numeric property, you can sum that particular property across all objects in the collection.

Let us see an example of using the LINQ Sum Method with Complex Type in C# using both Method and Query Syntax. We are going to work with the following Employee class. As you can see, it is a very simple Employee class with four properties: **ID**, **Name**, **Salary**, and **Department**. Here, we also created one method, i.e., **GetAllEmployees()**, which will return the list of all the employees, which will be our data source.

```
using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> listStudents = new List<Employee>()
            {
                new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
                new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
                new Employee{ID= 103,Name = "James", Salary = 50000, Department = "Sales"},
                new Employee{ID= 104,Name = "Hina", Salary = 20000, Department = "IT"},
                new Employee{ID= 105,Name = "Anurag", Salary = 30000, Department = "Sales"},
                new Employee{ID= 106,Name = "Sara", Salary = 25000, Department = "IT"},
                new Employee{ID= 107,Name = "Pranaya", Salary = 35000, Department = "Sales"},
                new Employee{ID= 108,Name = "Manoj", Salary = 11000, Department = "Sales"},
                new Employee{ID= 109,Name = "Sam", Salary = 45000, Department = "Sales"},
                new Employee{ID= 110,Name = "Saurav", Salary = 25000, Department = "Sales"}
            };
            return listStudents;
        }
    }
}
```

Now, our requirement is to calculate the Sum of the Salaries of all the Employees. The following example calculates the sum of all employees' salaries using the LINQ Sum method with both Method and Query Syntax. Here, we specify the numeric Salary column using a lambda expression to the Sum method.

```
using System;
using System.Linq;
namespace LINQDemo
{
```

```

class Program
{
    static void Main(string[] args)
    {
        //Using Method Syntax
        var TotalSalaryMS = Employee.GetAllEmployees()
            .Sum(emp => emp.Salary);
        //Using Query Syntax
        var TotalSalaryQS = (from emp in Employee.GetAllEmployees()
            select emp).Sum(e => e.Salary);

        Console.WriteLine("Sum Of Salary = " + TotalSalaryMS);
        Console.ReadKey();
    }
}

```

Output: **Sum Of Salary = 266000**

Example using Sum and Where Extension Method in C#:

Let us see an example of using both LINQ Sum and Where Extension Methods with Complex Type using Method and Query Syntax. Our requirement is to calculate the sum of the salary of all the employees who belong to the IT department. The following example exactly does the same. Using the Where Extension Method, we are filtering the IT department employees. Using the Sum method, we specify the Salary numeric column, which will calculate the sum of the salaries of only IT department employees.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Calculate the Sum of Salaries of IT Department
            //Using Method Syntax
            var TotalSalaryMS = Employee.GetAllEmployees()
                .Where(emp => emp.Department == "IT")
                .Sum(emp => emp.Salary);
            //Using Query Syntax
            var TotalSalaryQS = (from emp in Employee.GetAllEmployees()
                where emp.Department == "IT"
                select emp).Sum(e => e.Salary);
            Console.WriteLine("IT Department Total Salary = " + TotalSalaryQS);
            Console.ReadKey();
        }
    }
}

```

Output: IT Department Total Salary = 120000

Let's rewrite the previous example using the custom predicate. As you can see, within the Sum method, we are checking the Department property value, and if the Department is It, we are returning the corresponding Salary. Else, we are returning 0.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax and Predicate
            var TotalSalaryMS = Employee.GetAllEmployees()
                .Sum(emp => {
                    if (emp.Department == "IT")
                        return emp.Salary;
                    else
                        return 0;
                });

            Console.WriteLine("IT Department Total Salary = " + TotalSalaryMS);
            Console.ReadKey();
        }
    }
}
```

Output: IT Department Total Salary = 120000

Key points to remember when using the Sum Method:

- The Sum method calculates the sum of numeric values in a sequence or collection.
- The Sum method will throw an exception if the source collection is null. Therefore, it's important to ensure that the collection is not null before calling Sum.
- If the collection is empty, the Sum returns 0.
- For nullable types, if all elements are null or the collection is empty, Sum returns null.
- You can use the Sum method with various numeric data types, including integers, floating-point numbers, and custom numeric types.

When should you use the LINQ Sum Method in C#?

Here are some common scenarios where the Sum method is particularly useful:

- **Calculating the Total of Numeric Values:** The most straightforward use of Sum is when calculating the total of numeric values in a collection. For example, summing up all the numbers in an array or list.
- **Summing Based on a Selector Function:** You can use Sum with a lambda expression to calculate the sum of specific properties in a collection of objects. For instance, if you have a collection of objects

representing orders, and each order has a total amount, you can sum these amounts to get the total revenue.

- **Conditional Sum:** Combined with Where or similar filtering methods, Sum can be used to calculate the sum of values that meet certain conditions. For example, summing the values in a collection that is greater than a certain threshold.
- **Aggregate Properties of Objects:** When working with collections of objects, Sum can be used to compute aggregate values based on a specific property of the objects. For instance, calculating the total value of items in a shopping cart, summing the total weight of items in a shipment, or aggregating the total amount of memory used by a set of processes.
- **Data Analysis and Reporting:** In scenarios like data analysis, financial computations, or generating reports where summation of numerical data is a common task, Sum is extremely useful. For example, you might use it to sum up total sales, total hours worked, total energy consumption, or any other metric where aggregation is required.
- **Performance Considerations:** For large datasets, Sum can be an efficient way to compute a total, especially compared to manually iterating over a collection and accumulating a total.

In the next article, I will discuss the [LINQ Max Method in C#](#) with Examples. I explain the LINQ Sum Method in C# in this article with examples. I hope you understand the need for and use of the LINQ Sum Method with examples.